

Designing a First-Year Project Course to Engage Freshman Software Engineers: An Experience Report

Dean Hendrix, David Umphress, and James H. Cross II
Computer Science and Software Engineering
Auburn University
Auburn, AL 36266, USA
E-mail: hendrtd@auburn.edu

Abstract

This paper describes a software engineering project course for freshman. The course has been designed to engage and motivate software engineering students at the beginning of their matriculation, while at the same time providing solid software engineering content. Student teams use a simple software process based on Extreme Programming to complete a semester-long project in computer animation. Course motivation, structure, and implementation are described.

1. Introduction

Capstone project courses, normally required as part of the senior year plan of study, are standard components of most if not all software engineering curricula. Indeed, the ABET accreditation criteria for engineering programs specifies that curricula must culminate in a major design experience [1]. These capstone, or “culmination,” courses serve the significant purpose of creating a situation in which the student’s prior learning and experience will be coalesced and focused on engineering a solution for a real-world problem.

At Auburn University, our ABET-accredited software engineering program requires not only a capstone course during the senior year but also a “cornerstone” course during the freshman year. Requiring a software engineering project of freshmen presents both challenges and opportunities. Many freshmen have little or no background in programming or computer science from high school, thus making it difficult to create a meaningful project experience without making things too difficult or expecting too much from students who are just entering the program. If implemented well, however, a project-based introduction to software engineering course can provide freshmen with an early overview of the field as well as an “immersion” experience through participation in a fun, meaningful software engineering project.

2. Motivation and Context

The design of our freshman project course is motivated by several issues. Declining enrollments and student attrition are major concerns in computing programs throughout the United States. Although obviously related to each other, they are distinct issues. Enrollment in computer science programs has declined precipitously for the last five years [2, 3], and anecdotal evidence suggests that software engineering programs face a similar situation. Despite software engineering being listed as a top 10 growth job by the U.S. Bureau of Labor Statistics [4], students cite “cold feet” worries with respect to outsourcing and the dot-com demise [3]. Factors influencing student attrition are varied and include student preparedness in math and computing [5, 6, 7] and the belief that students have in themselves regarding their abilities to be successful in computing [8].

It is also important to note that the majority of students who leave computer science do so by the end of their freshman year [9].

To address the problem of attrition, we wanted to engage and motivate our software engineering students as early as possible. The literature documents various approaches to accomplishing this, including student retreats for personal development [10] and “crossover” team projects [11]. Our approach was to design a freshman project course (ENGR 1110 Introduction to Software Engineering) in which teams of students develop significant software applications in an engaging domain that would normally be out of the scope of their abilities.

Computer animation can be exciting to many students, and our own anecdotal evidence suggests that the desire to create video games and digital effects in movies is a significant factor in some students’ initial choice to major in computer science or software engineering. Computer graphics and animation require a high level of technical skill to create, certainly beyond the skill level of most software engineering freshman. Section 3 describes the use of Alice [12] to allow students with minimal knowledge of programming to create reasonably sophisticated computer games and movies (see Figure 1).

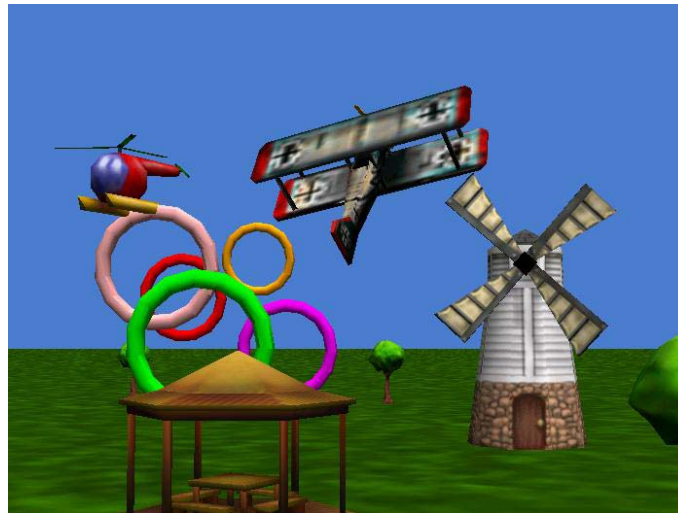


Figure 1. Screenshot from a computer game created with Alice.

While we wanted to provide a fun, motivating experience, we also wanted to ensure that students were exposed to solid software engineering content such as software process, software construction, teamwork, and technical communication [13]. Object-oriented software construction skills are developed with Alice, as discussed in Section 3. Students are required to complete some individual assignments, but most of the work in ENGR 1110 is accomplished in the context of a team. Students are assigned to teams on the basis of declared major (ENGR 1110 is open to all majors, not just software engineering), level of prior experience in programming, gender, and race. Every effort is made to make each team multi-disciplinary (major) and diverse (gender and race). Although there are published reports suggesting that some of these factors may not have a significant impact on student performance [14], our own experiences and anecdotal evidence suggests that participating in diverse teams is a beneficial experience for students.

Introducing software process early can help students establish good software practices instead of undisciplined habits, but can introduce many challenges [15, 16]. Drawing on lessons learned from the literature [17] as well as the adage “too much rigor leads to rigor mortis” – particularly with freshmen – we decided to create a simple adaptation of the popular agile methodology Extreme Programming (XP) [18] for ENGR 1110 teams.

Rather than creating a structured requirements document, students assemble simple user stories. Since the project is in the context of computer animation and gaming, students are able to see an intuitive correspondence between user stories and software functionality. There is also a clear mapping between user stories and the storyboards that are created during design.

Students go through a release planning phase in which the user stories are allocated to specific system releases. While the instructional staff specifies system release dates, it is the responsibility of each team to ensure that user stories are implemented appropriately for the releases. Iteration planning is the sole responsibility of the team. The just-in-time nature of iteration planning is a very nice fit for inexperienced freshmen. Most students seem to see this type planning as natural and effective rather than an as onerous administrative burden.

3. Engaging Students with Computer Animation

Alice is a programming environment that provides a high-level graphical user interface for creating animated 3D virtual worlds [12]. Alice is very easy to use, yet it is powerful enough to create relatively complex computer games and movies featuring a wide variety of 3D characters. Although computer animation itself is engaging, Alice provides particular appeal among several student demographic groups. Young men are typically attracted by the possibility of creating a video game, while young women and minority students are often attracted by the story-telling aspect of animation shorts and animated movies. ENGR 1110 student teams are given a choice of creating a game or an animation short.

Alice introduces fundamentals of object-oriented programming by making objects things that students can see and interact with (see Figure 2). Students build virtual “worlds” and populate them with objects that can behave independently as well as work in concert with each other. Each object comes with predefined internal data (e.g., position, size, opacity) and public methods (e.g., move, turn, rotate). User-defined methods can also be added to objects. Students create programs by invoking methods on various objects that are in the world.



Figure 2. Example objects available in the gallery.

All control structures (sequence, decision, and repetition) plus concurrency and event handling are supported and can be easily used in an Alice program. The Alice editor prevents syntax errors by having students write code using an intuitive drag-and-drop interface (see

Figure 3). Thus, students can concentrate on problem solving rather than learning syntactic rules that are so often frustrating to beginning programmers.

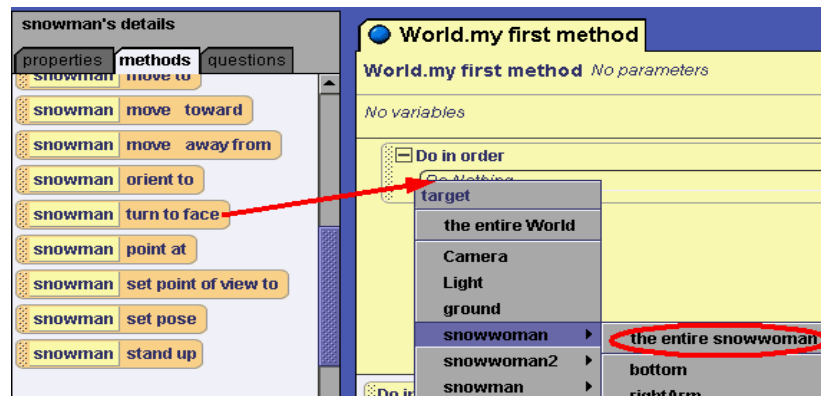


Figure 3. Drag-and-drop program editor.

ENGR 1110 students have used Alice with great success, and have created rather impressive projects for freshmen. Figures 4 through 6 provide screenshots from sample projects, including arcade style shooting gallery games (Figure 4) as well as console style games with sophisticated interaction for navigation, weapon targeting, shooting, and obstacle avoidance (Figures 5, 6). It should be noted that these projects require the students to address non-trivial issues such as concurrency, event handling, and collision detection techniques. Alice enables this level of challenge to be appropriate for first-semester freshmen with little or no prior experience with computing.



Figure 4. Sample project screenshot.



Figure 5. Sample project screenshot.



Figure 6. Sample project screenshot.

4. Evaluation and Summary

Evaluation of our approach to ENGR 1110 has only begun. Currently there is only anecdotal evidence, but it is uniformly positive. Students report that they enjoy using Alice, and in particular they enjoy being able to develop software that is fun, engaging, and something that they can be proud to show their friends. We have also noticed that students are able to engage in more complex problem-solving and engineering decision making earlier since we began using Alice in ENGR 1110. Rather than concentrating on learning a language, they are able to devote their energies to team dynamics, time management, process issues, and true engineering problem solving.

The use of Alice in computing curricula is not new, but its use has typically focused on teaching introductory programming concepts in a CS 1 or even CS 0 course [12, 18]. In that context, there is ample support for its effectiveness. In one experimental study, the use of Alice was shown to have a positive effect on at-risk students in early computing courses [12]. Specifically, using Alice prior to CS 1 improved at-risk students' course grade in CS 1 by one letter grade on average. Also, the retention rate for at-risk students who used Alice prior to CS 1 was 88%, almost twice that of at-risk students with no Alice exposure.

Our use of Alice is in contrast with what has been typically reported in the literature. Rather than focusing on teaching programming concepts, we focus on the broader software engineering concepts that are being taught in the course. Alice enables students to participate in non-trivial development projects while concentrating on issues such as process, design, and teamwork.

One issue that we have had to consider is the effect that using Alice in ENGR 1110 has on the content and approach that we use in our follow-on CS 1 course COMP 1210. After using Alice, student expectations can be difficult to meet in a traditional CS 1 course. After using so much visualization and computer animation, students come to expect something similar in the next course. Using a standard text editor and delving into Java syntax can be a let-down after Alice.

COMP 1210 is an objects-early CS 1 course that uses Java and is required of all software engineering majors. We use jGRASP (www.jgrasp.org) as the integrated development environment. jGRASP automatically generates many visualizations for Java such as control structure diagrams and UML class diagrams. The jGRASP object workbench, visual debugger, and dynamic object viewers allow students to work with visual representations of objects in Java in a way similar to how they worked with the inherently visual objects in Alice.

A more formal evaluation of ENGR 1110's effectiveness is planned for upcoming semesters. Current anecdotal evidence, however, clearly suggests that the course has met its goal of engaging and motivating software engineering students at the beginning of their curriculum, while at the same time providing solid software engineering content.

5. References

- [1] ABET Engineering Accreditation Commission, "Criteria for Accrediting Engineering Programs", <http://www.abet.org/>.
- [2] Jay Vegso, "CS Bachelor's Degree Production Grows in 2004; Poised for Decline", *Computing Research News*, Vol. 17, No. 2, March 2005.
- [3] Eric Chabrow, "By the Book", *Information Week*, August 16, 2004.
- [4] U.S. Bureau of Labor Statistics, <http://www.bls.gov>.
- [5] J.D. Davy, K. Audin, M. Barkham, and C. Joyner, "Student Well-Being in a Computing Department", *Proceedings of the 5th Annual Conference on Innovation and Technology in Computer Science Education*, Helsinki, Finland, 2000, pp. 136-139.
- [6] D. Hagan and S. Markham, "Does It Help to Have Some Programming Experience Before Beginning a Computing Degree Program?", *Proceedings of the 5th Annual Conference on Innovation and Technology in Computer Science Education*, Helsinki, Finland, 2000, pp. 25-28.
- [7] M. Morrison and T. Newman, "A Study of the Impact of Background and Preparedness on Outcomes in CS1", *Proceedings of the 32nd SIGCSE Technical Symposium on Computer Science Education*, Charlotte, NC, 2001, pp. 179-183.
- [8] T. Busch, "Gender Differences in Self-Efficacy and Attitudes Towards Computers", *Journal of Educational Computing Research*, Vol. 12, No. 2, pp. 147-158.
- [9] E. Seymour and N. Hewitt, *Talking About Leaving*, Boulder: Westview Press.

- [10] Mark Ratcliffe, John Woodbury, and Lynda Thomas, "Improving Motivation and Performance Through Personal Development in Large Introductory Software Engineering Courses", *Proceedings of the 15th Conference on Software Engineering Education and Training (CSEET 2002)*.
- [11] A.J. Cowling, "The Crossover Project as an Introduction to Software Engineering", *Proceedings of the 17th Conference on Software Engineering Education and Training (CSEET 2004)*.
- [12] Barbara Moskal, Deborah Lurie, Stephen Cooper, "Evaluating the Effectiveness of a New Instructional Approach," *Proceedings of the 35th SIGCSE Technical Symposium on Computer Science Education*, March 3-7, 2004, Norfolk, Virginia, USA.
- [13] R. Duley, G.W. Hislop, T.B. Hilburn, and A.E.K. Sobel, "Engineering an Introductory Software Engineering Curriculum", *Proceedings of the 16th Conference on Software Engineering Education and Training (CSEET 2003)*.
- [14] Pearl Brereton and Sue Lees, "An Investigation of Factors Affecting Student Group Project Outcomes", *Proceedings of the 18th Conference on Software Engineering Education and Training (CSEET 2005)*.
- [15] D. Suri and M.J. Sebern, "Incorporating Software Process in an Undergraduate Software Engineering Curriculum: Challenges and Rewards", *Proceedings of the 17th Conference on Software Engineering Education and Training (CSEET 2004)*, pp. 18-23.
- [16] David A. Umphress and John A. Hamilton, "Software Process as a Foundation for Teaching, Learning, and Accrediting", *Proceedings of the 15th Conference on Software Engineering Education and Training (CSEET 2002)*.
- [17] Anuja Shukla and Laurie Williams, "Adapting Extreme Programming For A Core Software Engineering Course", *Proceedings of the 15th Conference on Software Engineering Education and Training (CSEET 2002)*.
- [18] Kent Beck, *Extreme Programming Explained: Embrace Change*, Addison-Wesley, 1999.